EchoBrains

Python

Day-01 notes

- 1.what is programming language?

  it enables to communicate between humans and computers.

- 2.what are the types of programming languages?
- 1.lowlevel : machine language

  ex:0101001100111

- 2.mediumlevel : Assembly

  ex:hexadecimal code

- 3.highlevel : close to humans

  1.procedural : it following a procedure to make a program

  ex : c language

  2.object oriented: java,python,c#.net

- 3.what is translator?

it takes a program written in source code and converts in to machine (binary code).

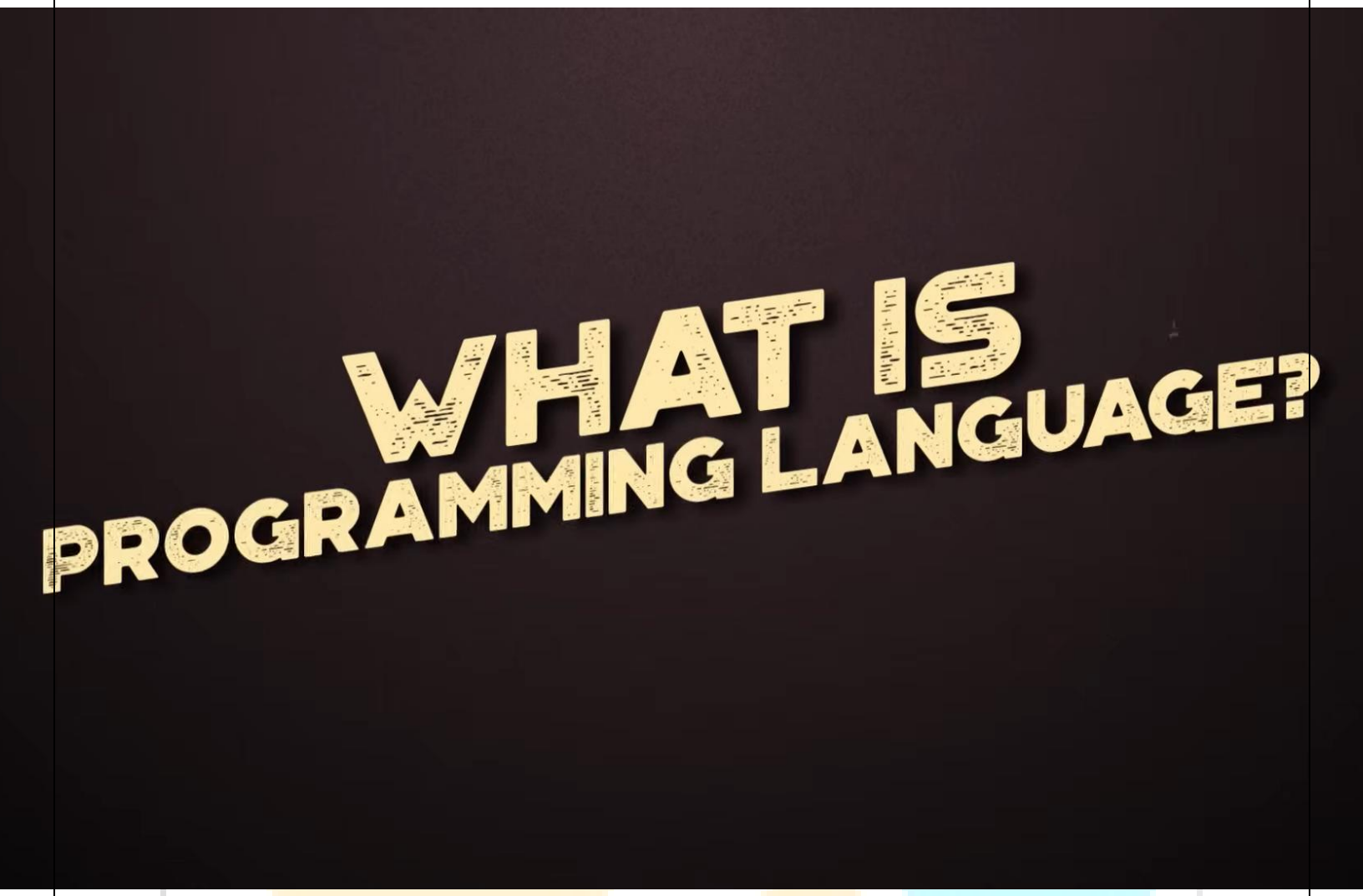
- 4.Types of translators?

  assembler : FAP,MAP(Macro)

  compiler : it check completely and it won't go for execution.

  interpreter : line by line execution.

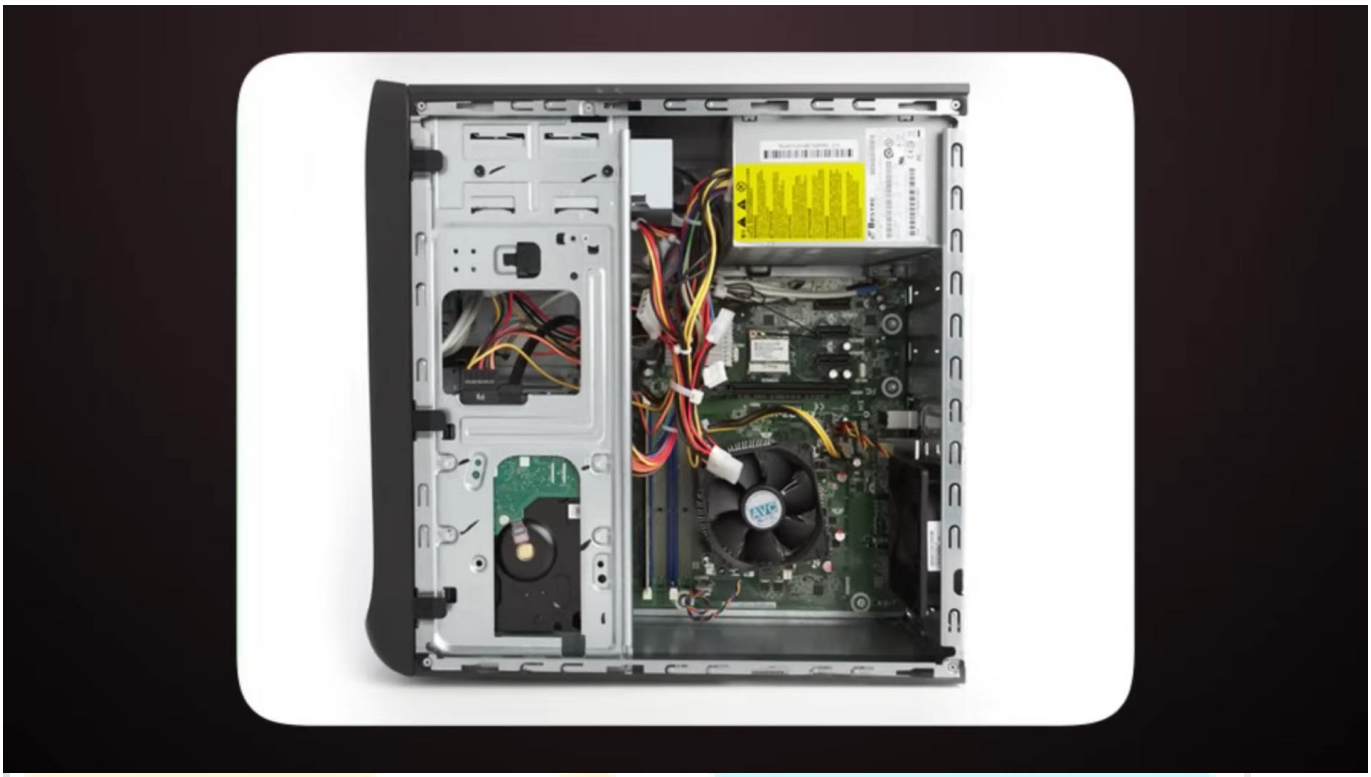
- 4.1 tools: idle , vscode , pycharm ,notepad,sublime,autom,jupyter notebook,colab.

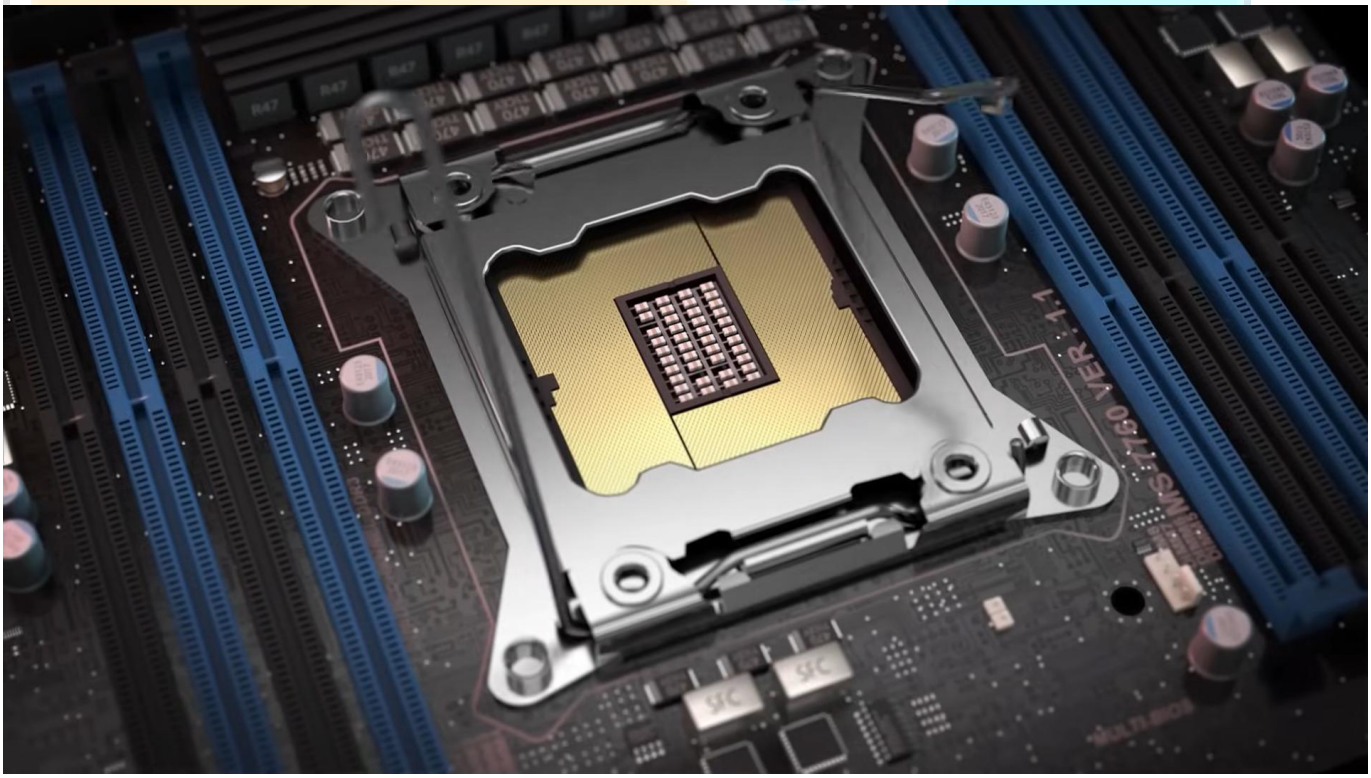

WHAT IS
PROGRAMMING LANGUAGE?

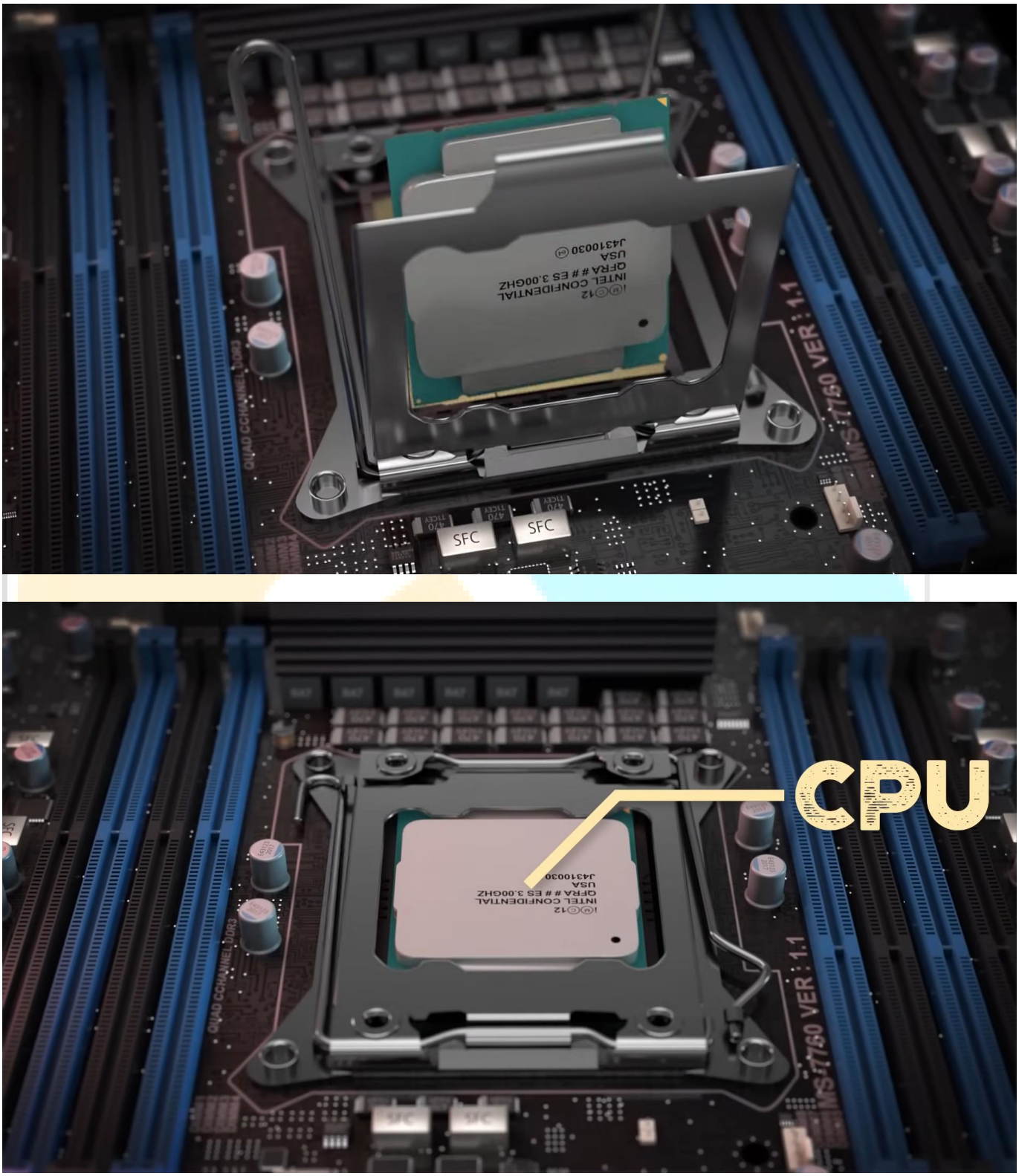
A computer processor, also known as the central processing unit (CPU), is the brain of a computer


CPU


CABINET



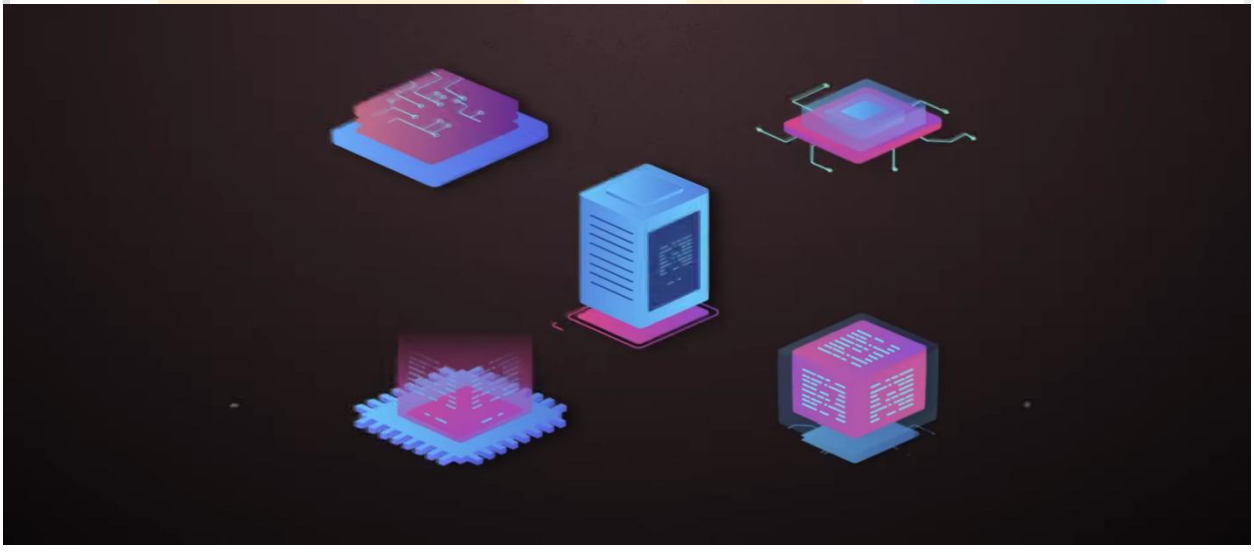


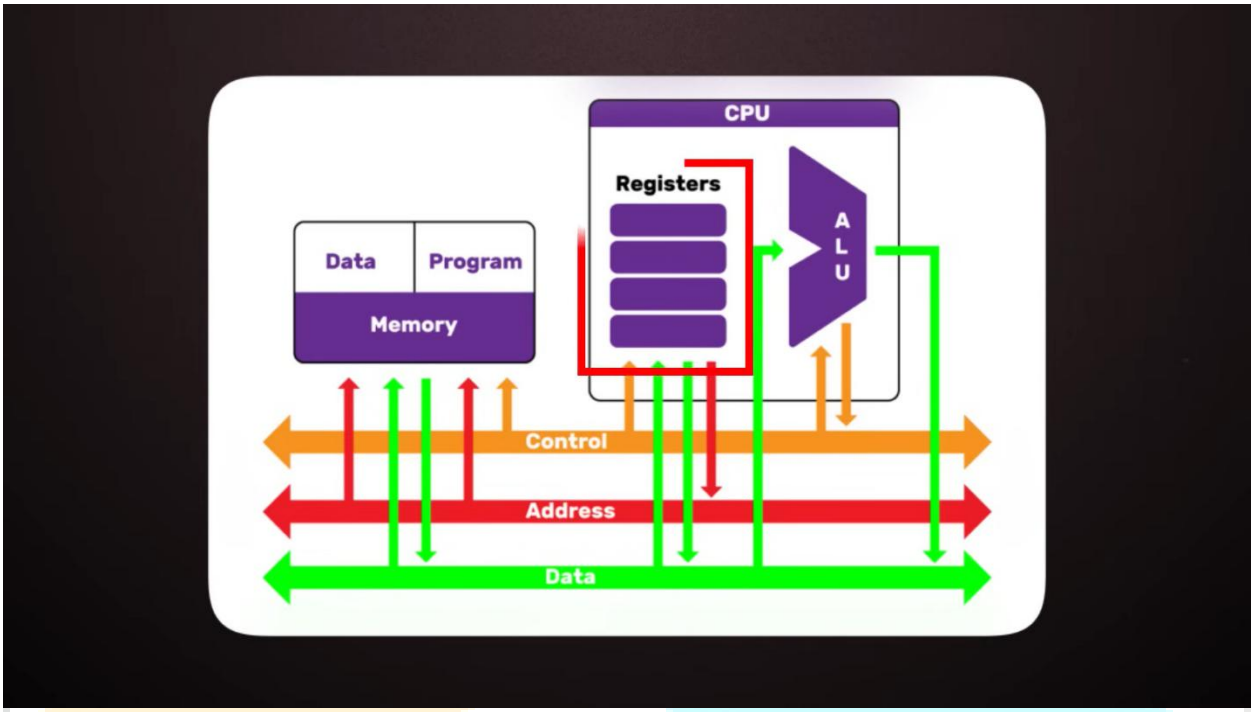

CPU

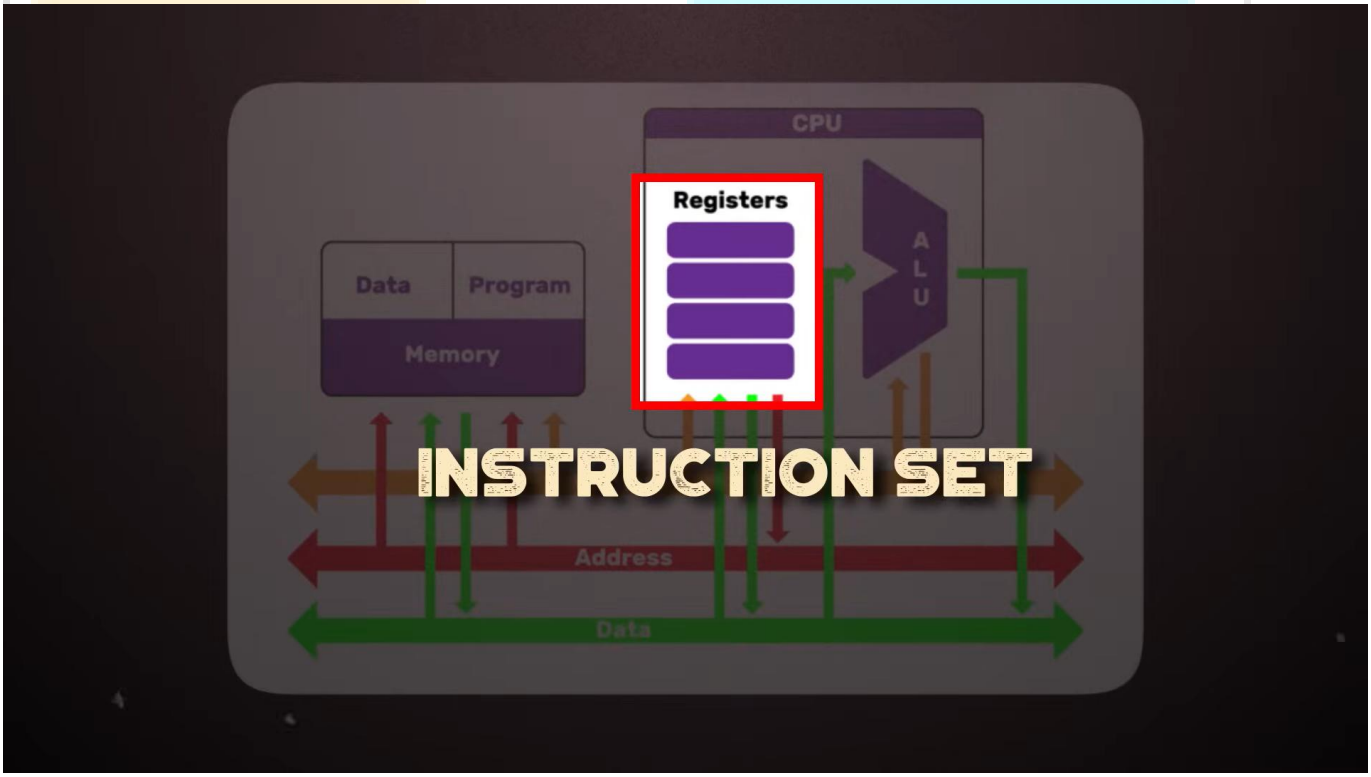
MATHEMATICAL CALCULATION

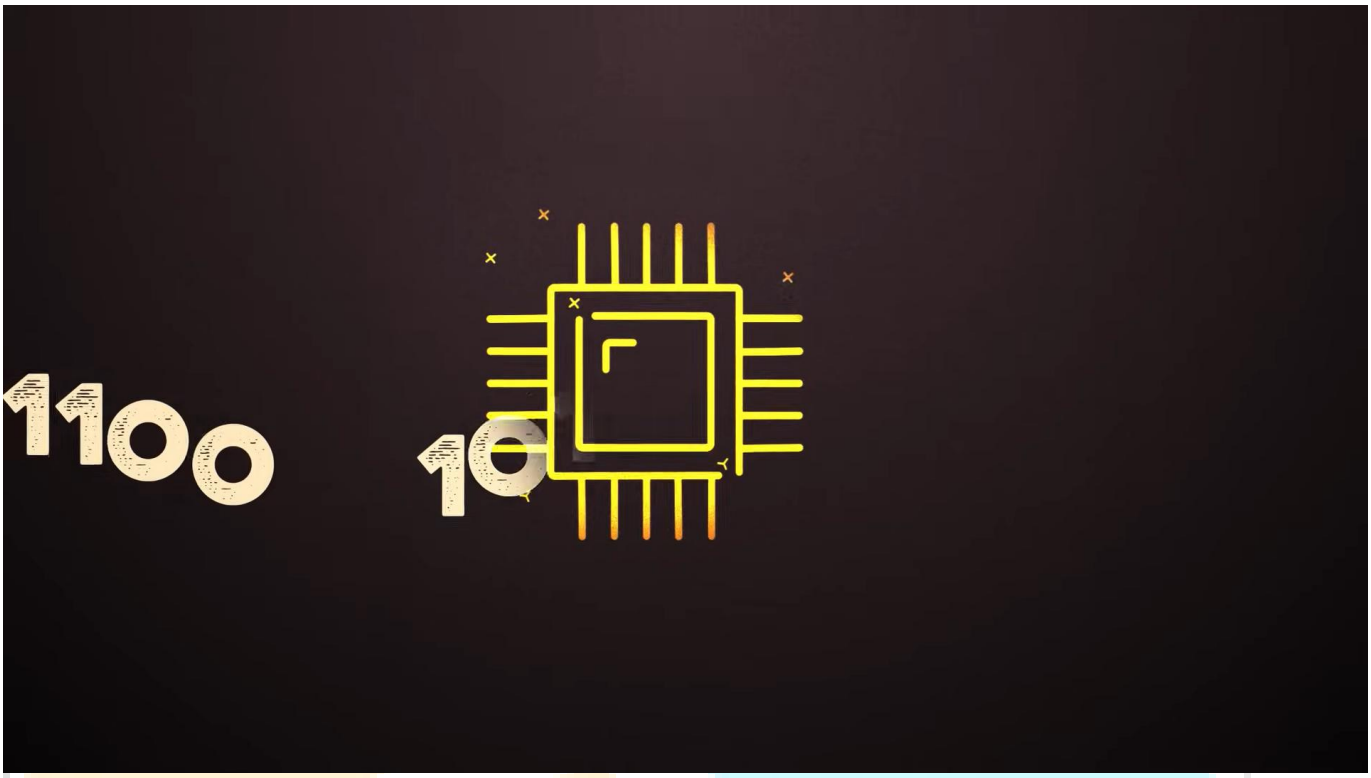
INPUT / OUTPUT MODULE

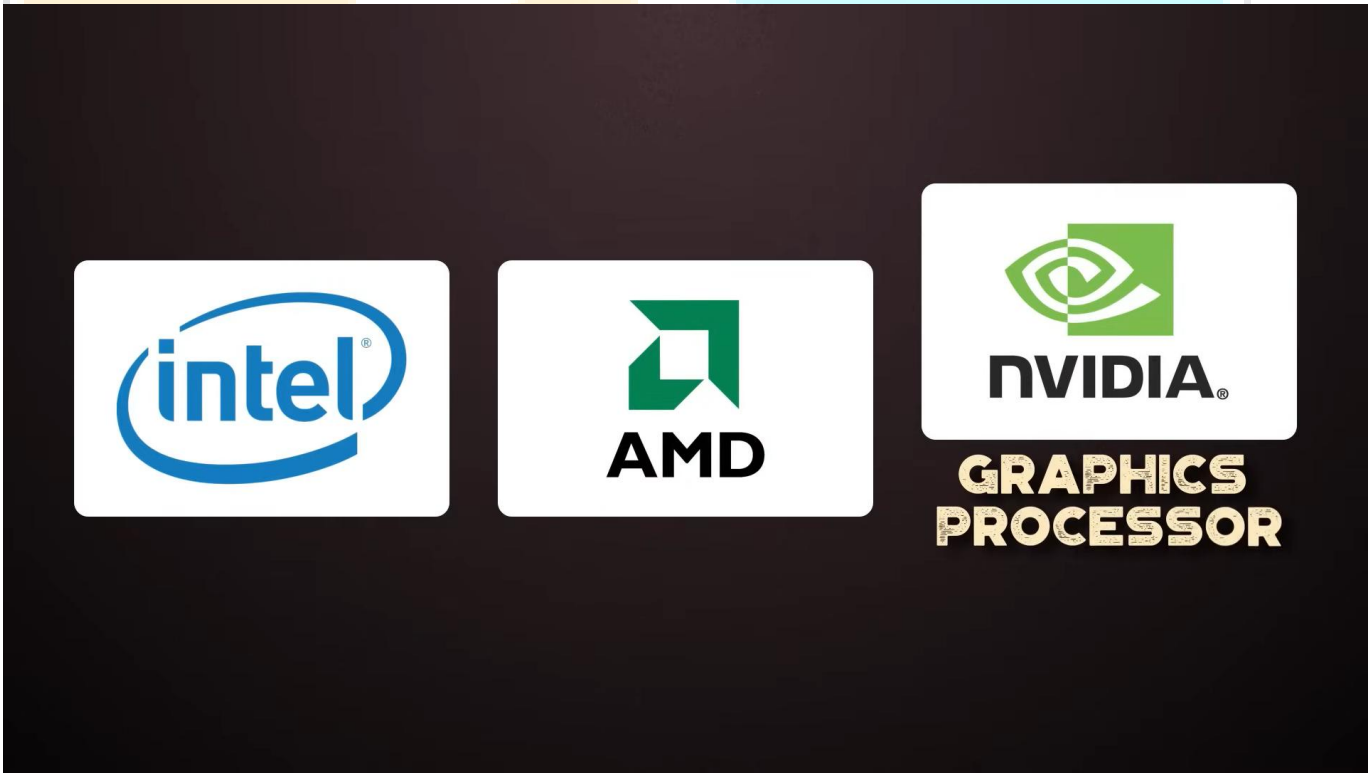
ARITHMETIC-LOGIC UNIT

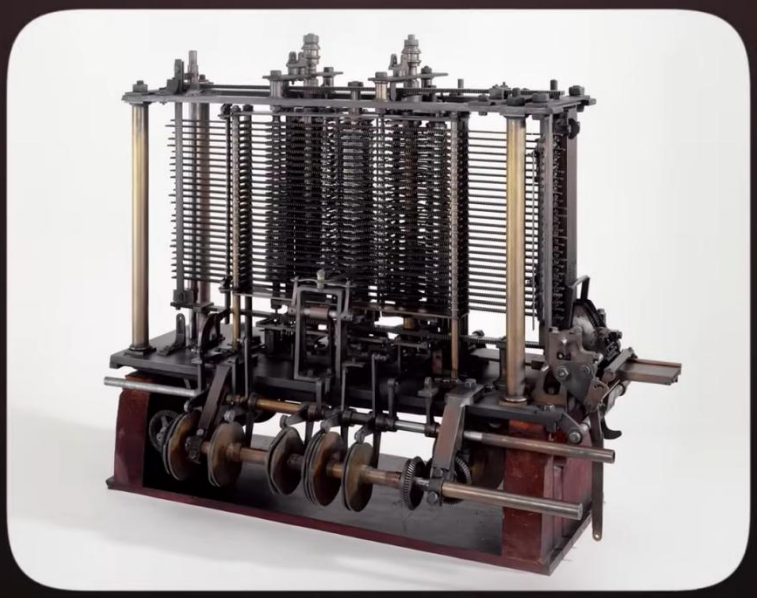



CPU
Registers
A
L
U
Data
Program
Memory
Control
Address
Data

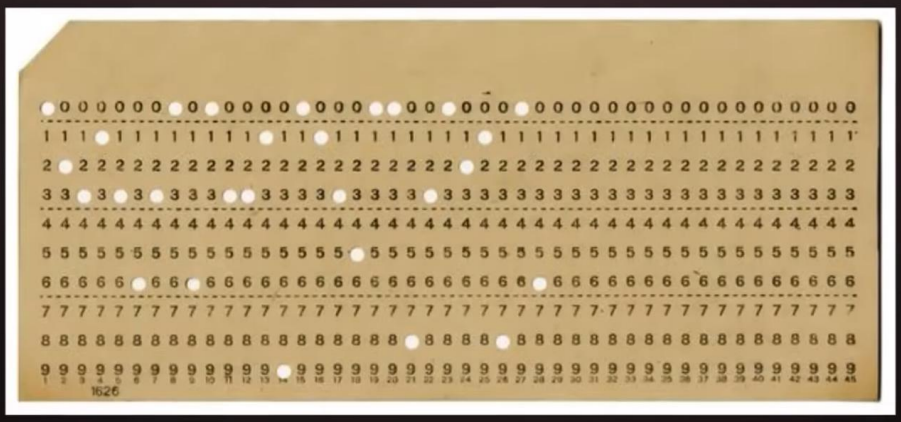

CPU
Registers
A
L
U
Data
Program
Memory
INSTRUCTION SET
Address
Data


1100
10


intel
AMD
NVIDIA
GRAPHICS
PROCESSOR



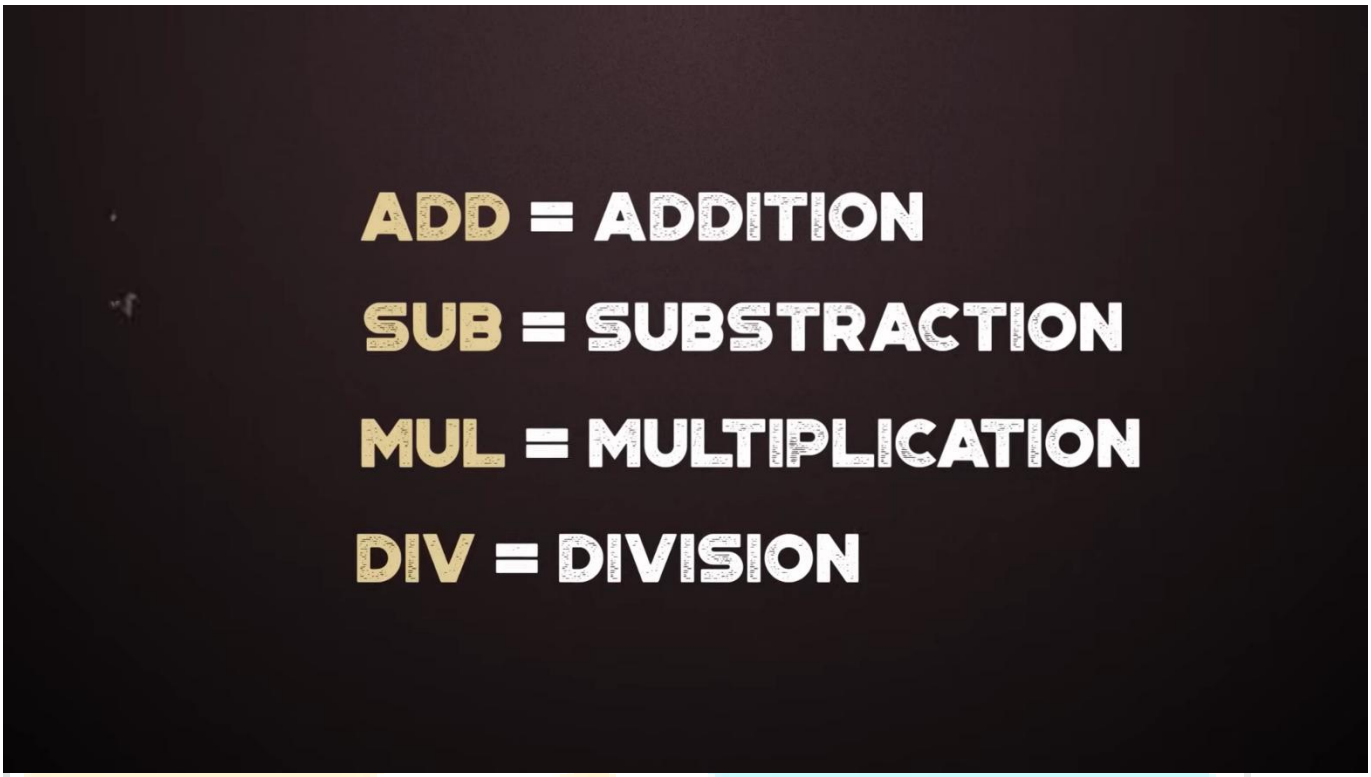
PUNCHED CARD



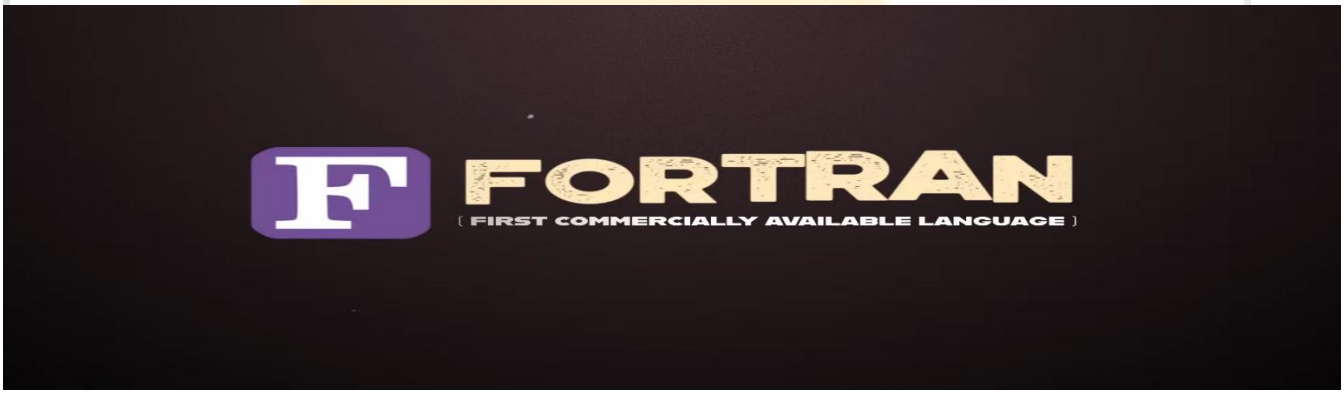
2ND GEN LANGUAGE

mov ecx, ebx
mov esp, edx
mov edx, r9d
mov rax, rdx

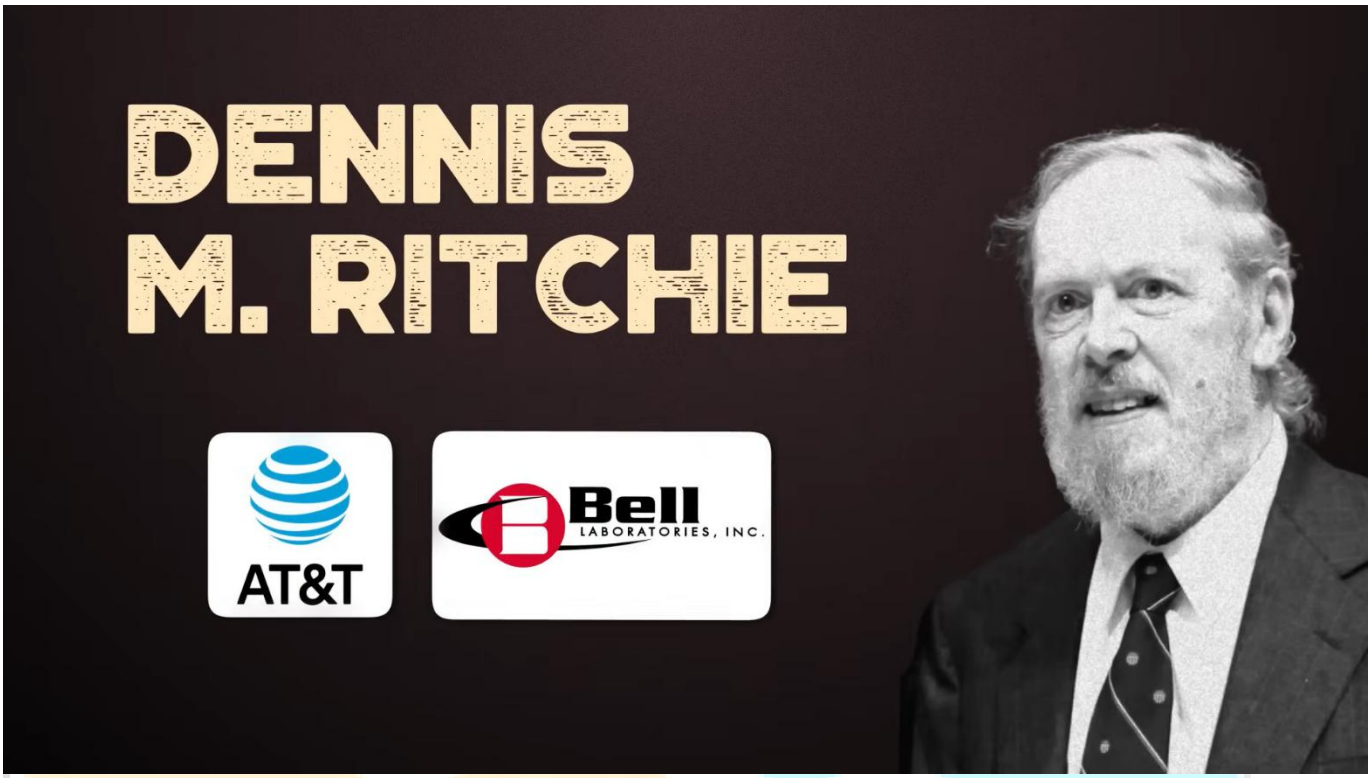
ASSEMBLY LANGUAGE

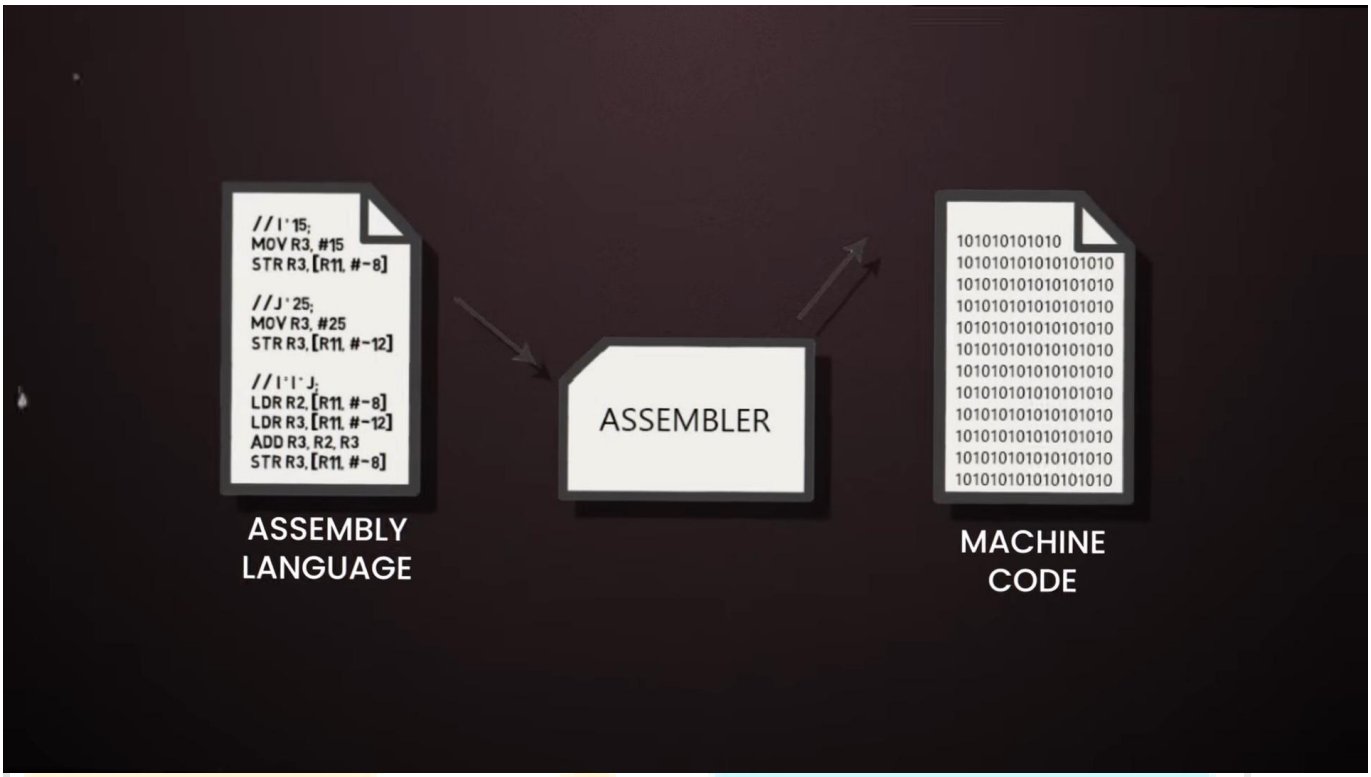


And 1940 -1950's

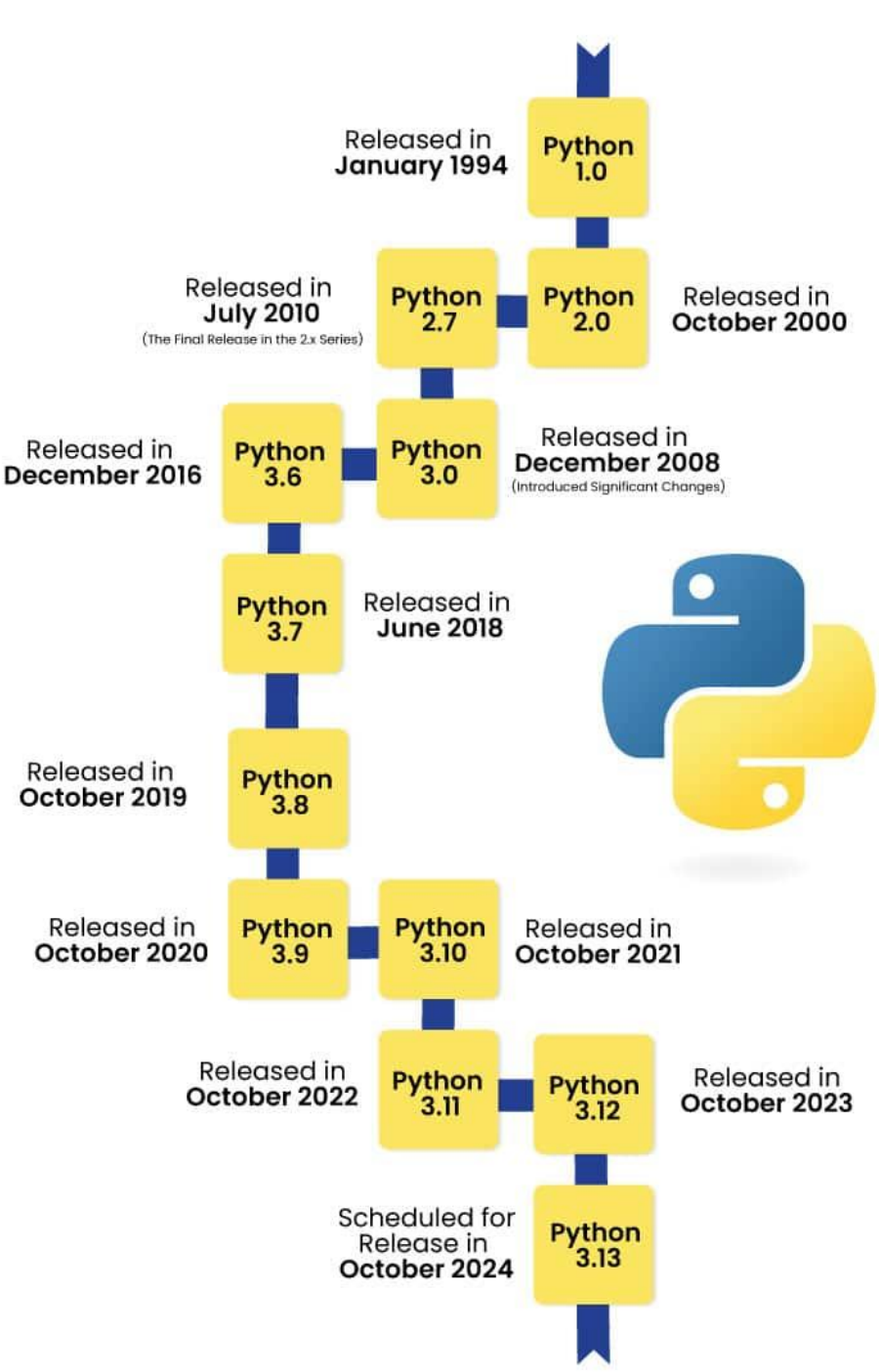


Then 1950 – 1960




//I = 15;
MOV R3, #15
STR R3, [R11, #-8]
//J = 25;
MOV R3, #25
STR R3, [R11, #-12]
//I = I + J;
LDR R2, [R11, #-8]
LDR R3, [R11, #-12]
ADD R3, R2, R3
STR R3, [R11, #-8]
ASSEMBLY LANGUAGE
ASSEMBLER
101010101010
1010101010101010
MACHINE CODE


INTEGRATED DEVELOPMENT ENVIRONMENT
eclipse
VS CODE


IntelliPaat
Released in January 1994
Python 1.0
Released in October 2000
Python 2.0
Python 2.7
Released in July 2010
(The Final Release in the 2.x Series)
Released in December 2008
(Introduced Significant Changes)
Python 3.0
Python 3.6
Released in December 2016
Python 3.7
Released in June 2018
Python 3.8
Released in October 2019
Python 3.9
Released in October 2020
Python 3.10
Released in October 2021
Python 3.11
Released in October 2022
Python 3.12
Released in October 2023
Python 3.13
Scheduled for Release in October 2024

Variables in Python

Variables in Python are fundamental building blocks, serving as containers for storing data values. In Python, memory space is allocated automatically without the need for explicit declaration when a value is assigned to a variable. The assignment of values to variables is done using the equal sign (=).

Example of Variable in Python

Python is dynamically-typed, which means you don't need to declare the type of a variable explicitly. Here's an example to illustrate how variables work in Python:

```
# Assigning a value to a variable
score = 100

# Assigning a string to another variable
player_name = "Alex"

# Assigning a list to another variable
items_collected = ['sword', 'shield', 'potion']

# Printing the values stored in variables
print("Score:", score)
print("Player Name:", player_name)
print("Items Collected:", items_collected)
```

Output:

```
Score: 100
Player Name: Alex
Items Collected: ['sword', 'shield', 'potion']
```

Variables Assignment

Python simplifies variable assignment, making it easy and intuitive for developers.

Creating Variables

In Python, creating a variable is as simple as assigning it a value. Python is dynamically typed, so you don't need to declare the type. The variable is created the moment you first assign a value to it.

```
x = 5
name = "John"
print(name)
```

Output:

```
John
```

Python Redeclaring Variables

You can redeclare a variable in Python by assigning a new value to it. This can even change the variable's type, due to Python's dynamic typing.

```
x = 5
x = "Sara"
print(x)
```

Output:

```
Sara
```

Initially, x is an integer. After redeclaration, x becomes a string.

Python Assign Values to Multiple Variables

Python allows assigning values to multiple variables in one line, making your code concise and readable.

```
a = b = c = 5
print(a)
print(b)
print(c)
```

Output:

```
5
5
5
```

Assigning Different Values to Multiple Variables

You can assign different types of values to multiple variables in a single statement in Python.

```
a, b, c = 5, 3.2, "Hello"
print(a)
print(b)
print(c)
```

Output:

```
5
3.2
Hello
```

Here, a becomes an integer (5), b a float (3.2), and c a string ("Hello").

Can We Use the Same Name for Different Types?

Yes, in Python, we can use the same variable name for different types. Due to Python's dynamic typing, the type of a variable is inferred from the value assigned, and the type can change as you reassign different values.

```
x = 100        # x is an integer
x = "Python"   # Now x is a string
print(x)
```

output:

```
Python
```

+ Operator with Variables

The + operator in Python can be used with variables for addition or concatenation, depending on the data type.

- For numbers, it performs addition:

```
a = 10
b = 20
print(a + b)
```

Output:

```
30
```

- For strings, it performs concatenation:

```
first_name = "John"
last_name = "Doe"
print(first_name + " " + last_name)
```

Output:

```
John Doe
```

Introduction

In this article, we will understand the basic syntax of Python.

What is Python Syntax?

Before we move on to understanding python syntax, let's understand how to write and run a basic Python program.

There are two ways to do so:

1.Interactive mode – In this mode, you write and execute the program

2.Script mode – In this mode, you first create a file (.py file), save it, and then execute it.

Enter the following command in your terminal after installing Python to enter the interactive mode.

$ python

You have now entered the interactive mode. You can run any of the Python statements in the interactive mode in the terminal now. But if you're using a script editor / IDE, you do not have to do the same.

Now we can move on to understand the python syntax. When we learn the Python syntax, we first start from the basic syntax of python like studying the print statement.

Print Statement

Let us run the Python first program in the terminal now

```
>>> print("Basic Syntax of Python")
```

The output for the print statement is:

```
Basic Syntax of Python
```

Now we discussed the interactive mode above, what about the script mode? To run code in the script mode, you must first create a .py file, write your code and save it. You can write the same code as above (the print statement). Let's say you saved it as syntax.py. Once you've done that, you can either use the 'run' button in your IDE or if you would like to run it on your terminal :

```
$ python syntax.py
```

```
Basic Syntax of Python
```

Once you execute the command, you see that it prints – Syntax of print on the terminal. Congratulations! You've learned your first basic Python syntax.

Check out this article to know more about print() function in Python.

Indentation in Python

Whenever we write any piece of code in Python, say, functions or loops, we specify blocks of code for them. How will we identify which block is for what? It is done by "indenting" statements of that block according to the Python syntax.

To determine the line's indentation level, we use a number of leading whitespaces, which could be spaces or tabs. The General notion is to use one tab (or four spaces) for a single indent level.

Let’s take a look at an easy example to understand indentation levels.

```
def foo_bar():
    print("Hello!")
    if True:
        print("Bye")
    else:
        print("See you soon")


print("Indentation in Python")
```

We will not go into the code and what it does; instead, let’s focus on the indentation levels.

```
def foo_bar():
```

It marks the beginning of the function, so every line of code that belongs to the function has to be indented by one level at least. Notice that the print statements and the if statements are indented by at least one level. But what about the last print statement? As we see, it is indented by 0 levels, which means that it is not part of the function.

Let’s now move inside the function block, which is every statement below the function beginning that is indented by at least one level.

The first print statement is indented by one level because it is only under the function and not any other loop or condition. In contrast, the second print statement comes under the if statement.

```
if True:
    print("Bye")
```

The if statement is indented by just one level, but any block of code that has to be written under it should be indented by one more level than the if statement. The same goes for the else statement.

Now, in the indentation of code in Python, we have to follow specific rules. They are :

Indentation cannot be split into multiple lines with the use of the backslash (‘\’) character

An IndentationError will be thrown if you try to indent the first line of code in Python. You cannot indent the first line of code.

If you have indented your code either by using a tab or space, it’s recommended to continue with the same spacing preference throughout your code. Do not use a mix of tabs and whitespaces to do so, as it may cause the wrong indentation.

It is best to use 1 tab (or 4 whitespaces) for the first level of indentation and continue adding 4 more whitespaces (or 1 more tab) for higher indentation levels.

Indenting code in Python has benefits like making the code look beautiful and structured. It gives you a good understanding of the code in a single look. Plus, the indentation rules are simple, and if you’re writing code on an IDE, then most IDEs would automatically indent the code for you.

One disadvantage of indentation is that if your code is extensive and involves high indentation levels, even an indentation error in a single line can be very tedious to fix.

Let us look at some indentation errors to make your understanding crystal clear.

```
>>>     y = 9
  File "<stdin>", line 1
    y = 9
IndentationError: unexpected indent
>>>
```

As discussed above, since we cannot indent the first line, and Indentation Error is thrown.

```
if True:
    print("Inside True")
     print("Will throw error because of extra whitespace")
else:
    print("False")
```

It will throw an IndentationError because the second print statement is written with one level indent, but it has extra whitespace, which is not allowed in Python.

```
if True:
print("Will throw error because of 0 indent level")
else:
print("Again, no indent level so error")
```

In the code above, the print statements inside the if and else are not indented by one extra level, and hence IndentationError will be thrown.

```
IndentationError: expected an indented block.
```

Python Syntax Identifiers

When we talk about variables, functions, classes, or modules, we use “identifiers” to identify them. What are identifiers? It is a name given to something to differentiate it from other code entities.

Now just like for indentation, we have some naming rules here.

The identifier will contain a combination of lowercase (a-z) or uppercase(A-Z) alphabets or numbers (0-9) or underscore (_)

The Identifier cannot start with a digit

Any reserved words or keywords cannot be used as identifier names (you’ll read more about keywords in the article soon)

Symbols or Special characters cannot be used in the identifier

Length of the identifier can be variable; there is no restriction

Remember that Python is a case-sensitive language, so var and VAR are two different identifiers

Examples of correct naming: var, Robot, python_007, etc

Examples of incorrect naming: 97learning, hel!o etc

Some identifier examples for variables:

```
var = 310

My_var = 20

My_var344 = 10

STRING = "PYTHON"

fLoat = 3.142
```

Some examples of functions :

```
def my_function():
    print("Valid")


def My_function():
    print("Valid")


def Myfunction3():
    print("Valid")


def FUNCTION():
    print("Valid")
```

To check if your name is a valid identifier or not, you can use the inbuilt function “isidentifier()”.

It can be used as :

```
print("var".isidentifier())

print("@var".isidentifier())
```

```
True

False
```

The first one is a valid identifier; hence it will print True, and the second one is invalid and will print False.

There is a catch with this function, though. You cannot use any keywords to check if they’re valid identifiers or not. This function will show True for any keyword, but you know that keywords cannot be used for identifiers. Hence, here this function has a caveat.

Now we discussed keywords and reserved words multiple times in this section, let’s take a look at them.

Python Keywords

Listed below in the table are the keywords in Python. They are some unique purpose words that can be used only for specific cases and not as identifiers.

False None True

__peg_parser__ and as

assert async await

break class continue

def del elif

else except finally

for from global

if import in

is lambda nonlocal

not or pass

raise return try

while with yield

To view all the keywords your version of python supports, you can open your script editor or IDE or write the following two commands on your terminal to list all the keywords.

```
import keyword

print(keyword.kwlist)
```

```
['False', 'None', 'True', 'and', 'as', 'assert', 'async', 'await', 'break', 'class', 'continue', 'def', 'del', 'elif', 'else', 'except', 'finally', 'for', 'from', 'global', 'if', 'import', 'in', 'is', 'lambda', 'nonlocal', 'not', 'or', 'pass', 'raise', 'return', 'try', 'while', 'with', 'yield']
```

Python Variables

What are variables? To store any data values, we use containers called variables. There is no specific command in Python to declare a variable. A variable is created the moment a value is assigned to it.

For example:

```
y = 9

x = "Foo"

print(x)

print(y)
```

It will give us the output –

```
Foo

9
```

In most languages, you are required to declare variables with a particular type, as in the type of value they hold. It could be int, char, etc. But this is not required in Python. You can declare a variable ‘x’ as an integer (by assigning an integer value) and then change it to any other type as a string (by assigning a string value).

The variable names given follow the rules of identifiers that we discussed above.

Click here, to know more about Variables in Python.

Comments in Python

First, let’s understand why we need comments or why commenting in a program is essential. Of any program, comments are an integral part. We can make multiple-line comments as docstrings or single-line comments.

Writing comments is essential when you're reading your code. It makes the code simpler to understand. You may even make specific comments in between to check and edit certain blocks of code. Or even later, say after six months, you have to make some changes in your code. The comments that you make throughout would help you understand your code.

Say you're working in a team. You understand the code that you've written and do not require comments. But it would be challenging for others to understand your code if you don't explain it in short lines as comments.

Now that we know why comments are critical let's understand how we can include them in our Python programs.

To write a simple single-line comment, you must add the # symbol before your comment. Like this :

```
# This is a comment
```

You need not add it on a new line. You can even add it after statements in your code.

```
print("This is the statement")    # This is a comment explaining the statement
```

According to the Python syntax, anything that is written after the # symbol is ignored by Python. It will not run.

Now say you want to write multiple comments together, as in a multiline comment, you cannot do it directly in Python. Most languages just use /* … */ for multiline comments.

```
# incorrect way to write multi-line comments in Python
```

```
/*

so this is

just not

possible in Python

*/
```

However, you can do this :

```
# this is

# one way to

# write multiline comments in python
```

If you don’t want to constantly hit enter and add the # symbol at the start of every line, you can make it a docstring by adding three double quotation marks.

```
"""

It makes multiline

comments in Python

much easier

"""
```

Let me also give you a short trick on commenting on multiple lines of code all at once. Select the lines of code that you want to convert into comments, and press Ctrl + / together in Windows or Command + / in Mac. That would make your job much easier!

That’s all you need to know about comments in Python!

Multiple Line Statements

Before understanding multiline statements, let us take a look at statements.

```
x = 10  # this is a statement (1)



class FooBar:  # this is statement (2)

    pass  # statement number (3)
```

Each of these lines is a statement. What if some statements are lengthy and you cannot fit them properly in one line? That’s when multiline comments come in handy.

Using the continuation character backslash, you can explicitly divide a statement into multiple lines (“\”).

For example

```
message = "This is a long statement that I want to write, but the " \

        "problem is that it doesn't fit in one single line. So this is how " \

        "I've written it for readability."
```

Did you also know that we can write multiple statements in one line?

```
a = 1; b = 2; c = 3
```

The following three statements have been written in a single line using the semicolon (;).

Quotation in Python

Three types are used when it comes to quotation marks accepted by Python according to the Python syntax.

Single – ' ' Double – " " And Triple – " " " … " " "

We have already seen the usage of the triple quotation marks in the comments section. Single and double quotations are used to declare strings in Python.

Blank Lines in Python

There are three ways to print blank lines in Python according to the Python syntax.

The first and simplest way is to use a blank print statement:

```
print("Line # 1")

print()

print("Line # 3")
```

Output:

Line # 1

Line # 3

Another way is to put empty quotation marks in the print statement, single or double.

```
print("Using single or double quotation marks")
print('')
print("")
print("Works!")
```

Output:

Using single or double quotation marks

Works!

The third way is to use a newline character (\n) at the end of the statement:

```
print("Using newline\nto create a new line!")
```

Output:

Using newline

to create a new line!

Waiting for User

Taking input in the Python programming language is done by the input() function. The program with input() function will not run till the program is not provided input by the user through the console.

Example –

```
print("Enter name")

name = input()
```

After the second line, the program waits for the user to enter input.

Command Line Arguments

To keep our program's nature generic, we use command-line arguments. For example, if we have written a program to parse a CSV file, inputting a CSV file from the command line would make our program work for any CSV file, making it generic.

Command-line arguments also make our program more secure. How? We need to put in some login credentials in the code to use them somewhere. If we add them to the script, they may be accessed by anyone and even executed. But on the other hand, if we use the command line to enter the login credentials, it becomes safer.

So how do we pass command-line arguments in Python?

It’s easy. You need to run the Python script from the terminal the way we discussed at the beginning of the article and add the inputs after that.

```
python code.py arg1 arg2 … arg N
```

Here, code.py is the script's name, and arg1 … argN are the N number of arguments required as input from the command line.

You must be wondering how one would read the command line arguments? The Common way is to use the sys module for it.

Sys module

The sys module has a variable named argv. All the command line arguments are stored as a list of strings.

If you want the first command-line argument entered, you can access it by argv[0].

Example:

```
import sys


if len(sys.argv) != 3:
  raise ValueError("Please provide your name and age")


print(f' Your name is {sys.argv[1]} and your age is {sys.argv[2]}')
```

The first line of the program is just importing the sys library. In the second line, we're using an if statement to check the length of the user input. Since we're asking for two values (name and age), we need precisely two values in the command line input. If it is lesser or greater than that, the program is set to raise an error.

In the print statement, you can see that we have used the sys.argv[i] to fetch and print the input.

To run this program, in the terminal, we write :

python script.py Laila 23

Laila and 23 are the two command-line inputs we're giving to the python program.

Its output will be:

Your name is Laila, and your age is 23

If you don't enter two arguments, you will see a value error.

It is the most commonly used module; however, there are more modules such as getopt or the argparse module.

FAQs

Q. What is the Python syntax for comments?

A: In Python, you can add comments to your code using the hash character (#). Anything written after the hash symbol on the same line is considered a comment and is ignored by the Python interpreter. Comments are useful for adding explanations or notes to your code. Here's an example:

```
# This is a comment in Python

print("Hello, World!")  # This is another comment
```

Q. What is the significance of the if _name_ == "_main_": statement?

A: The if _name_ == "_main_": statement is often used in Python scripts. It checks whether the script is being run directly as the main program or if it's being imported as a module. The code inside this block will only execute if the script is the main program. This is useful when you want to include test code or specific actions that should only run when the script is executed directly.

Q. How do you declare and use variables in Python?

A: In Python, you can declare a variable by assigning a value to it using the equals (=) operator. Unlike some other programming languages, you don't need to explicitly specify the variable type. Python dynamically determines the type based on the value assigned to it. For Example:

```
# Variable declaration and assignment

name = "John"

age = 25

height = 1.75
```

```
# Variable usage

print("Name:", name)

print("Age:", age)

print("Height:", height)
```

Conclusion

There are 2 ways to write and run a Python program: interactive mode and script mode.

Python Syntax for the print statement is – print(‘Syntax of print’)

After every line of code that ends with ‘:’, python expects an indented code block. It is usually done with 4 whitespaces or 1 tab.

Your code can throw an indentation error if there’s extra whitespace, an unexpected indent, or no indent.

Identifiers in Python are used to identify variables, classes, functions, or objects. Some correct naming methods are – var, Robot, python_007, etc.

Python has 36 keywords which are essentially reserved words that cannot be used as identifiers.

Variables are containers that can be used to store any values. Variable names follow the same rules as identifiers.

In python, you can use the ‘#’ symbol to write single-line comments like this :

this is a comment

or write multiline comments like this:

"""

This is multiline.

Written in 3 quotations.

"""

If a statement is too long, it can be broken down using the '\' character and written in multiple lines. Alternatively, you can write multiple short statements in one line by terminating each statement with the ';' semicolon.

In Python, three types of quotation marks are used. The single and double quotation marks are for strings, while the triple quotation marks are for multiline comments according to the Python syntax.

The three ways to add a blank line in Python are:

Use the empty print statement.

Use print statements with empty single or double quotation marks

Use the newline character (\n)

The input() function allows us to wait for user input. The program will not proceed further without user input according to the Python syntax.

Using the sys module, we can take command-line arguments

See Also:

Assert Keyword in Python.

What is Assert?

An assert statement can be used to verify any logical assumptions you make in your code. It is mainly used as a debugging tool. For example, if we write a function that performs division, we know that the divisor must not be zero, so we "assert" that the divisor shouldn't be equal to zero so as to prevent any kinds of errors/bugs that could occur due to this problem in the code later on.

As input, the assert in Python takes an expression that is to be evaluated, and [an error message, which is optional. Basically, when the code encounters any assert statements, one of two things can happen.

One, the code runs without any issues; two, it throws an AssertionError, terminating the program. We are going to delve deep into these situations, but we must first understand why exactly the assert statement should be used.

Why Use Assert in Python?

An assert statement in Python recognizes issues right off the bat in your program, where the reason is clear, instead of later when some other operation fails.

They can be termed as internal self-checks for your code. The aim of using the assert statements is so that developers can track the root cause of the bug in their code quickly.

Assert statements particularly come in handy when testing or assuring the quality of any application or software.

Python Assert Statement

Syntax:

```
assert <condition> , <error message>
```

As stated above, we have a condition and an error message as part of the assert statement. If the condition used in the assert statement evaluates to True, the code runs normally i.e. without throwing errors or terminating the program. On the other hand, if it evaluates to False (unwanted condition) it throws an AssertionError along with the error message, if provided and terminates the program.

Assert in python,

```
assert <condition>
```

is almost similar to this:

```
if __debug__:
    if not <condition>:
        raise AssertionError()
```

What is __debug__ here? It is a constant that is used by Python to determine if statements like assert must be executed.

In the Python programming language, whether we use the assert statement, or raise the error using if, it does not have any major differences, they're almost similar. The code with if raises an AssertionError when the debug mode is on (__debug__ == True), and the condition provided evaluates to False.

When we can use an if statement, which is definitely easier to understand, **why** boggle minds with the assert statement? There are two main reasons – readable code, and the option of disabling the assert statements if required.

Using the assert statement instead of the if condition makes the code more readable and shorter. An advantage of using the assert statement over the if condition is that an assert statement can be disabled, unlike an IF condition.

To disable any assert statements in the program, running Python in optimized mode (__debug__ == False), ignores all assert statements. The -O flag can be passed for the same.

```
python -O program.py
```

Now that we have seen the syntax of the assert statement, as well as its advantage over if, let us move on to see what it looks like in the Python shell, and how we run it:

```
>>> assert True   # nothing, because <condition> is True
>>> assert False  # will raise an error but no message, since not specified
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
AssertionError
>>> assert False, "This statement is False"  # condition is False, hence
error + message, since it is specified

Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
AssertionError: This statement is False
```

If the assertion fails, a message can be printed like this as done in the shell example above :

```
assert False "This assertion has failed"
```

Not adding the error message does not make any difference in the way the program runs. The error displayed will be the Traceback error as shown in the example above and terminates the program. It just helps understand the assertion error more clearly.

Assertion errors in Python terminate the program you have written, but, what if we did not want that to happen? What if we want the code to run as-is, and along with it, we would like the error to be detected?

The solution for this problem is the try-except block. The try-except block in Python is another means of handling and checking errors in Python. First, only the code that is written inside the try block will get executed when there isn't any error in the program and the except clause is skipped. While in the presence of an error in the try block, the code in the except block will be executed and code in the try block will be skipped.

Here's how we handle a basic division by 0 assertion error using the try-except block:

```
try:
  x = 1
  y = 0
  assert y != 0 , "Division by zero not possible!"
  print(x / y)
except for AssertionError as msg:
  print(msg)  # error message given by user gets printed

print("The rest of the program still runs!")
```

Let's try and understand what happened in the above code. We used an assert statement, that, as we read above prints an error message upon evaluation of the condition to False, and we also print a msg in the except block. What are we doing here, and why?

When we're in the try block, we have values of x as 1 and y as 0, and an assert condition that prevents the division of any number by 0. Now since the value of y is 0, our assert statement evaluates to False and hence throws an AssertionError. We know that these errors terminate the program, but since we do not want it to terminate, we use the try-except. As soon as there is an error in the try block, we immediately move to the except block to handle the errors.

When we write a simple assert statement with a condition and an error message, what gets printed? The error message. So, the except block

```
>>> assert(2 + 3 == 6, "Incorrect addition")  # wrong
>>> assert 2 + 3 == 6, "Incorrect addition"   # correct
```

The first statement with the parentheses will not work because you're essentially providing the assert statement with a tuple. The format of a tuple in Python is (Val1, Val2, …) and any tuple written in the context of bool is always true unless it does not contain any values. This means that if we do not provide any condition to the assert statement, like :

```
assert()
```

it will raise an AssertionError because empty tuples are False.

What Does the Assertion Error do to our Code?

Upon reading and understanding multiple times that unhandled assertions terminate programs, a question comes to mind – How will it terminate the program?

There are essentially two ways of stopping the execution of a program – by exit() and by abort().

When a program is terminated by using exit(), the function performs – flushing of unwritten buffered data, closing of open files, and removal of temporary files and it returns an integer exit status to the operating system.

The abort() function may not close any files that are open, may not flush stream buffers, and may also not delete any temporary files. One major difference between both kinds of functions that terminate a program is that abort() results in abnormal termination while exit() causes normal termination.

Termination of the program due to an assert statement occurs with the help of the abort() function when the condition mentioned in the assert statement returns False.

Finally, now that we know what assert is, why it is useful, and how we can tackle it, we must address the elephant in the room. Where exactly can you use it?

Where do We Use Assert in Python?

Assertions are not used to flag expected error conditions, similar to “file not found” where a user can make a restorative move (or simply attempt once more). Instead, the assert statement is used mainly for debugging purposes,

1. Checking parameter types / classes / values

The assert statement can be used if your program has the tendency of controlling every parameter entered by the user or whatever else

Example:

```
def kelvin_to_fahrenheit(Temp):
   assert (Temp >= 0),"Colder than absolute zero!"  # User input cannot
violate a fact, hence the use of assert
   return ((Temp - 273) * 1.8) + 32
```

2. Checking “can’t happen” situations

While the assert statement is useful in debugging a program, it is discouraged when used for checking user input unless it corrupts a universal truth. The statement must only be used to

identify any “this should not happen” situation i.e. a situation where any fact, or a statement that is definitely True, becomes False due to a bug in the code.

For example, if the user enters fewer characters than required in the input, then it cannot be termed as a “this should not happen” situation, where the use of the assert statement is a must because the user isn’t violating any fact that is meant to be True always. It is not a blunder to use an assert statement for trivial issues such as this, but it is considered a bad practice.

```
def something(s: str):  # a function that takes a long string as the input to do something
  assert len(s) >= 8  # bad practice, the length of the input string doesn't violate a universal truth

  # do something
```

However, if say, a program involves calculations with radius and diameter of a circle and the radius of the circle isn’t half of the diameter at any point in the program, then this comes under the “this should not happen” case, as it is a fact that cannot be changed or violated.

For example,

```
# some complex operations involving the radius of a circle

assert radius == diameter / 2, "Calculations have corrupted the value of radius!!!"

# return statements with radius or continuation in the program
```

If during the complex operations, the value of the radius changes and is unequal to half of the diameter, our assert statement condition immediately detects it and throws an error so as to avoid any further complications/errors in the code or provide unwanted output.

3. Documentation of Understanding

Using the assert statement in your code is useful for documenting the programmer’s understanding of the code. It makes the programmer’s assumptions clear for themselves, as well as anyone else who works on the same code in the future. If for any reason, in the future the code gets broken, the reason for failure will be obvious, and the next time the assert condition evaluates to False, the caller will get the AssertionError exception.

Another Example of Assert in Python

Say you have a function that calculates the cost of an item after a discount. In this situation, we can positively say that the discounted cost will definitely be greater than 0 and lesser than the original cost.

If the discounted cost doesn’t lie in this range, then we are in a “this should not happen” situation. Hence, we use the assert statement.

```
def calculate_discount(cost, discount):
    discounted_cost = cost - [discount * cost]
    assert 0 <= discounted_cost <= cost  # used an assert because a fact could have been violated, and also serves as a depiction of the understanding of the programmer

    return discounted_cost
```

- Assertions or assert in Python help you state facts with confidence in a program, it helps verify logical assumptions in any code
- Syntax for assert statement in Python:

```
assert <condition> , <error message>
```

What is Python Programming ?

What is python ?

Single-line comments in Python.

Difference between Python 2 and Python 3

What is Python 2?

Python 2 is an older version of the Python programming language, released in the year 2000. It introduced many features that helped shape the language, such as Unicode support, garbage collection, and list comprehensions. Python 2 was known for its simplicity and ease of learning, making it popular in educational and development settings. However, its official support ended in 2020, which means it no longer receives updates or security fixes.

Example:

```
# Hello World program in Python 2.x

print "Hello World!\n"
```

Output:

```
Hello World!
```

What is Python 3?

Python 3 is the currently and actively supported version of Python, first released in 2008. It was designed to rectify fundamental design flaws in Python 2 and improve the language's capabilities. Key improvements include better Unicode support, a more consistent syntax, and enhanced library functions. Python 3 encourages more efficient and sustainable code practices. It is the recommended version for all Python development due to its ongoing support and advanced features.

Example:

```
# Hello World program in Python 3.x

print('Hello World!')
```

Output:

```
Hello World!
```

Python 2 vs. Python 3: What's the Difference?

Here's the table that illustrates difference between Python 2 and 3.

Comparison Parameter	Python 2	Python 3
Release Year	Debuted in 2000.	Debuted in 2008.
print Functionality	The print is a statement.	The print is a function.
String Storage	ASCII is the default string type.	UNICODE is the default string type.
Integer Division	Results in an integer quotient.	Results in a floating-point quotient.
Exception Syntax	Exceptions use 'notations'.	Exceptions use 'parentheses'.

Comparison Parameter	Python 2	Python 3
Global Variable Leak	Global variables can leak inside loops.	Global variables are stable inside loops.
Iteration Method	Uses the xrange() for iterations.	Introduces the range() function for iteration.
Syntax Simplicity	Has a more complex syntax.	Has a simpler and more intuitive syntax.
Library Compatibility	Libraries are less forward-compatible.	Libraries are designed to be strictly compatible with Python 3.
Current Usage	No longer in use since 2020.	More popular and actively used today.
Backward Compatibility	code can be ported to Python 3 with effort.	Not backward compatible with Python 2.
Primary Use	Was mainly used for DevOps engineering. No longer in use post-2020.	Widely used in diverse fields such as Software Engineering, Data Science, etc.

Python 2 vs. Python 3 Example Code

This below code explains the major difference between python 2 and 3: **Python 2**

```
# Python 2 code
# Print statement
print "Hello, World!"

# Integer division
result = 5 / 2
print result  # Output: 2

# xrange function
for i in xrange(5):
    print i,

# Input function
name = raw_input("Enter your name: ")
print "Hello, " + name

# Handling Unicode strings
str = "Hello, World! \u00A9"
print str

# Error handling
try:
    result = 10 / 0
```

```
except ZeroDivisionError as e:
    print "Error:", e
```

Output:

```
Hello, World!
2
0 1 2 3 4 Enter your name: John
Hello, John
Hello, World! ©
Error: integer division or modulo by zero
```

Python 3

```
# Python 3 code
# Print function
print("Hello, World!")

# True division
result = 5 / 2
print(result)  # Output: 2.5

# Range function (no xrange in Python 3)
for i in range(5):
    print(i, end=' ')

# Input function (renamed to input)
name = input("Enter your name: ")
print("Hello, " + name)

# Handling Unicode strings
str = "Hello, World! \u00A9"
print(str)

# Error handling
try:
    result = 10 / 0
except ZeroDivisionError as e:
    print("Error:", e)
```

Output

```
Hello, World!
2.5
0 1 2 3 4 Enter your name: John
Hello, John
Hello, World! ©
Error: division by zero
```

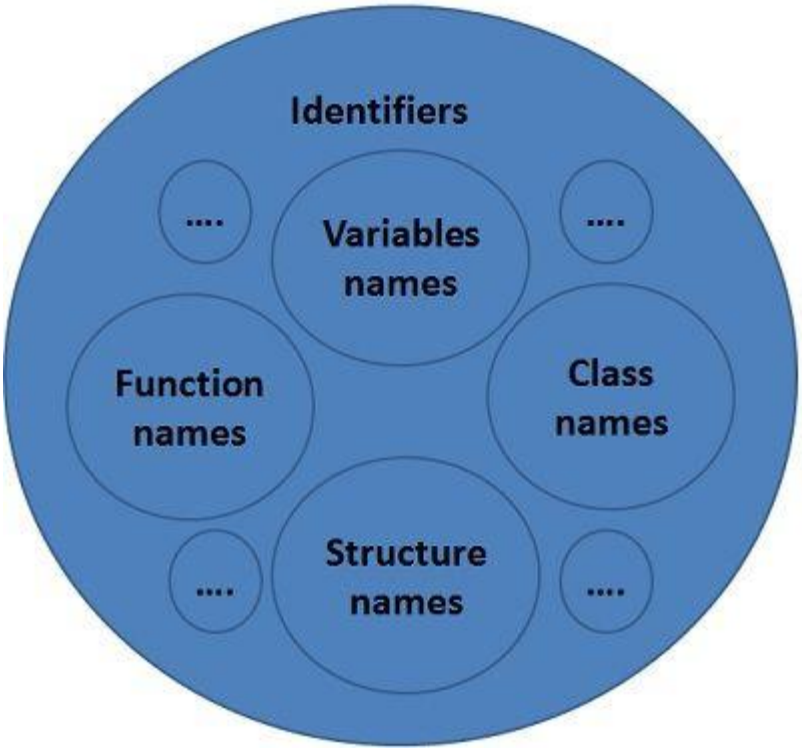
This example demonstrates the difference between Python 2 and Python 3, including the print statement/function, integer division, range/xrange functions

Python Identifiers

In Python, identifiers are names used to identify a variable, function, class, module, or other object. They are essentially user-defined names to represent these entities in a program. The purpose of identifiers is to make the code readable and meaningful, as they can describe the role of the entity they are naming.

Or

An identifier **is the name used to identify variables, functions, classes, modules, and objects in Python.**



Rules for Naming Python Identifiers

1. **Start with Letters or Underscore**: Identifiers must begin with a letter (A-Z, a-z) or an underscore (_). They cannot start with a digit.
2. **Case Sensitive**: Python identifiers are case-sensitive. This means myVar, MyVar, and myvar are three different identifiers.
3. **Contain Alphanumeric Characters and Underscores**: After the first letter or underscore, identifiers can contain letters, digits (0-9), and underscores.
4. **No Special Characters**: Identifiers cannot contain special characters like @, $, !, etc.
5. **Avoid Keywords**: Identifiers should not be any of Python's reserved keywords. For example, you cannot name an identifier if or for.
6. **Length**: There is no length limit for Python identifiers. However, keeping them within a reasonable length is good practice to maintain readability.
7. **Conventions**: Python follows certain naming conventions like using lowercase for variable names, uppercase for constants, and camelCase or snake_case for multi-word identifiers. Python does not enforce these, but they are good practices.

Examples of Python Identifiers

Valid Python Identifiers:

- **Variables**: height, max_value, _data
- **Functions**: calculate_area(), get_info()
- **Classes**: User, StockPrice
- **Constants**: MAX_LIMIT, PI

Invalid Python Identifiers:

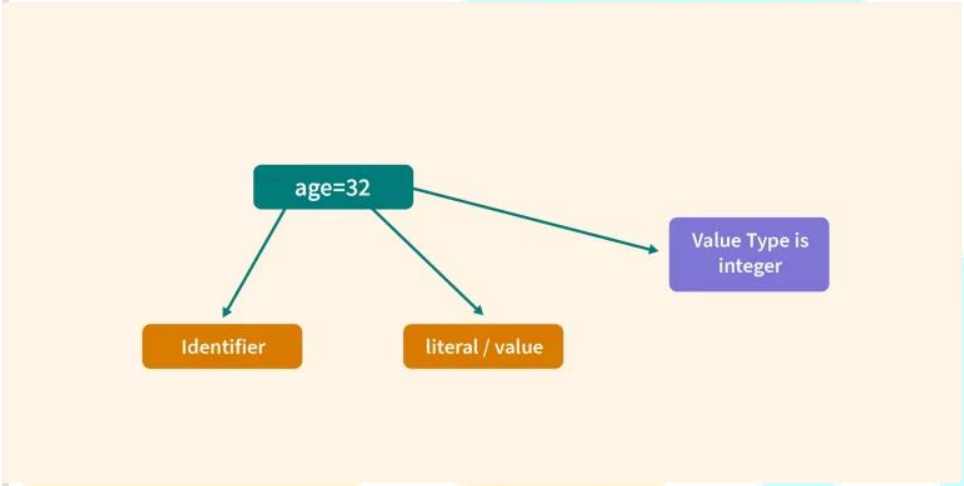
- **Starting with a digit**: 1variable (invalid)
- **Containing special characters**: user-name (invalid)
- **Using Python keywords**: for, if (invalid)

These examples follow the naming rules and conventions for Python identifiers. Identifiers are a critical part of the code as they give meaning to the different parts of the program, making it easier to understand and maintain.

Python Literals

Definition:

A literal **in Python is a fixed value assigned to a variable. It represents constant values directly in the code.**



Literal Type	Example
String Literal	`"Hello"` or `'World'`
Integer Literal	`100, -20`
Float Literal	`3.14, 2.0`
Complex Literal	`3 + 5j`
Boolean Literal	`True, False`
Special Literal (`None`)	`None`
List Literal	`["apple", "banana"]`
Tuple Literal	`("red", "green")`
Dictionary Literal	`{"name": "Alice", "age": 21}`
Set Literal	`{1, 2, 3, 4, 5}`

Q.How can you create a multi-line string literal in Python?

a)By using single quotes (')
b)By using double quotes ("")
c)By using triple quotes (""")
d)By using a backslash at the end of every line

Python Comments

They are just meant for the developers to understand a piece of code. Python interpreter completely ignores comments in Python.

1)single line comment:

Example:

```
#Welcome
```

2. multiline comment

Example:

```
"""
If you do not want to type # every time for
every line, you can make use of
delimiters!
"""
```

Example2:

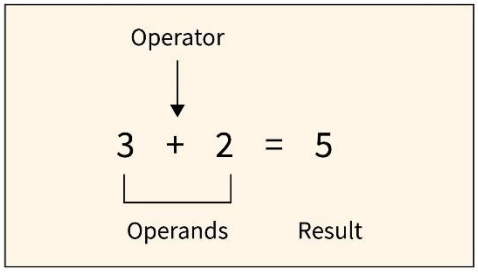
```
#Welcome
#to
#Scaler
```

#Academy

Operators in Python

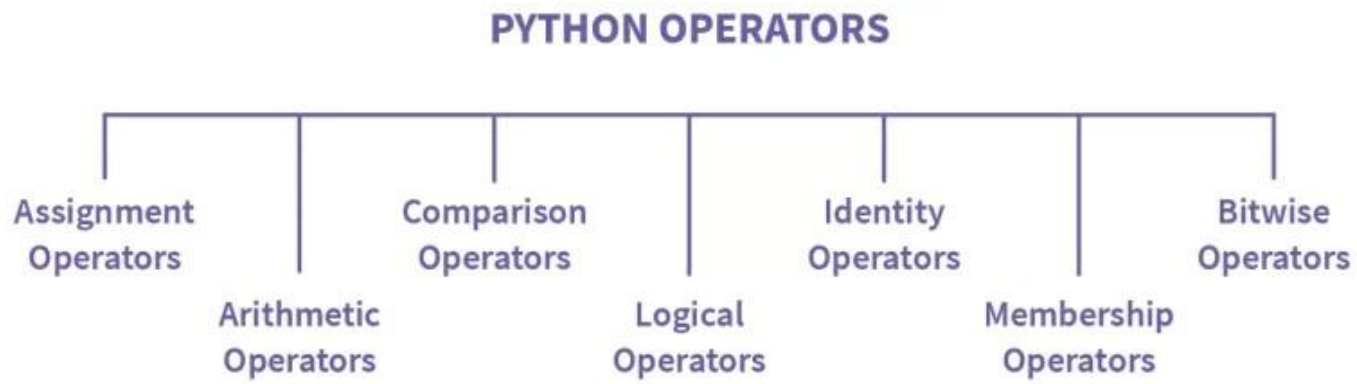
Definition:

Operators in Python are **symbols** that perform operations on **variables and values**.



Types of Operators in Python

Operators are divided into 7 categories in Python according to their type of operation.



Let's see each one of them in detail:

1. ARITHMETIC OPERATORs in Python

Arithmetic operators are what we have used in our school life. There are 7 types possible under arithmetic operations:

Operator	Operation	Example	Description
+	Addition	a+b	Adds a and b
-	Subtraction	a-b	Subtracts b from a
*	Multiplication	a*b	Multiplies a and b
/	Division	a/b	Divides a and b
%	Modulus	a%b	Gives remainder after dividing a from b
**	Exponential	3**2=9	Ignores the decimal value if present
//	Floor Division	15/2=7	Gives the result of 3 to the power of 2

2. ASSIGNMENT OPERATORs in Python

These are operators used for assigning values to a variable. The values to be assigned must be on the right side, and the variable must be on the left-hand side of the operator. There are various ways of doing it. Let's see each one of them in detail:

Operator	Description	Example
=	Assigns right side operands to left variable.	>>>'number'=10
+=	Added and assign back the result to left operand.	>>>'number'+=20 # 'number'='number'+20
-=	Subtracted and assign back the result to left operand.	>>>'number'-=5
*=	Multiplied and assign back the result to left operand.	>>>'number'*=5
/=	Divide and assign back the result to left operand.	>>>'number'/=2
%=	Taken modulus (Remainder) using two operands and assign the result to left operand.	>>>'number'%=3
**=	Performed exponential (power) calculation on operators and assign value to the left operand.	>>>'number'**=2
**=	Performed exponential (power) calculation on operators and assign value to the left operand.	>>>'number'**=2
**=	Performed floor division on operators and assign value to the left operand.	>>>'number'//=3

3. COMPARISON OPERATORs in Python

These operators compare the value of the left operand and the right operand and return either True or False. Let's consider a equals 10 and b equals 20.

Operator	Description	Example
==	If the values of two operands are equal, then the condition becomes true.	(a==b) is not true.
!=	If the values of two operands are not equal, then the condition becomes true.	(a!=b) is true.
<>	If the values of two operands are not equal, then the condition becomes true.	(a<>b) is true. This is similar to != operator
>	If the values of left operand is greater than the value of right operand, the condition is true.	(a>b) is not true.
<	If the values of left operand is less than the value of right operand, the condition is true.	(a<b) is true.
>=	If the values of left operand is greater than or equal to the the value of right operand, the condition is true.	(a>=b) is not true.
<=	If the values of left operand is less than or equal to the the value of right operand, the condition is true.	(a<=b) is true.

4. LOGICAL OPERATORs in Python

Let's understand the logical operator with an example. You decide you go to school only if your friend also goes. So either you both go to school or you both don't. And that's how an and statement works. If one of the conditions gives False as output, no matter what the other conditions are, the entire statement will return false.

Now in the or statement, if any one of the statements is True, the entire statement returns True. For example, you go to school if your friend comes or you've got a bike. So either of the statements can be True.

The not operator is used to reverse the result, i.e., to make False as True and vice versa.

These operators in Python are used when different conditions are to be met together. There are various types in it:

Operator	Example	Description
and	x < 5 and x < 10	Returns True if both statements are true
or	x < 5 or x < 4	Returns True if one of the statements is true
not	not(x < 5 and x < 10)	Reverse the result, returns False if the result is not true

5. IDENTITY OPERATORs in Python

These operators compare the left and right operands and check if they are equal, the same object, and have the same memory location.

Operator	Example	Description
is	x is y	Returns True if both variables are the same object
is not	x is not y	Returns True if both variables are not the same object

Example code:

```
number1 = 5

number2 = 5

number3 = 10

print(number1 is number2)  # check if number1 is equal to number2
```

```
print(number1 is not number3)  # check if number1 is not equal to number3
```

6. MEMBERSHIP OPERATORs in Python

These operators search for the value in a specified sequence and return True or False accordingly. If the value is found in the given sequence, it gives the output as True; otherwise False. The not in operator returns true if the value specified is not found in the given sequence. Let's see an example:

Operator	Example	Description
in	5 in {1, 2, 3, 4, 5}	It returns true if it finds the corresponding value in a specified sequence and false otherwise.
not in	5 not in {1, 2, 3, 4}	It returns true if it does not find the corresponding value in a given sequence and true otherwise.

7. BITWISE OPERATORs in Python

These operators perform operations on binary numbers. So if the number given is not in binary, the number is converted to binary internally, and then an operation is performed. These operations are generally performed bit by bit.

Let's consider a simple example considering a = 4 and b = 3. At first, both the values are considered in binary format, so 4 in binary is 0100, and 3 in binary is 0011. The bit-by-bit operation is performed when the & operation is performed.

So the last digits of both the numbers are compared, and it returns 1 only if both the numbers are 1; otherwise, 0. Likewise, all the bits are compared one by one and then provide the answer.

Operator	Name	Description
&	AND	Sets each bit to 1 if both bits are 1
|	OR	Sets each bit to 1 if one of two bits is 1
^	XOR	Sets each bit to 1 if only one of two bits is 1
~	NOT	Inverts all the bits
<<	Zero fill left shift	Shift left by pushing zeroes in from the right and let the leftmost bits fall off
>>	Signed right shift	Shift right by pushing copies of the leftmost bit in from the left, and let the rightmost bits fall off

Example code:

```
number1 = 4  # 0100 in binary

number2 = 3  # 0011 in binary

print("& operation- ", number1 & number2)  # AND

print("| operation- ", number1 | number2)  # OR

print("^ operation- ", number1 ^ number2)  # XOR

print("~ operation- ", ~number2)  # negation

print("<< operation- ", number1 << 1)  # shift left by 1 bit

print(">> operation- ", number1 >> 1)  # shift right by 1bit
```

output:

```
& operation-  0

| operation-  7

^ operation-  7

~ operation-  -4

<< operation-  8

>> operation-  2
```